A Decision Tree and a Pegasos SVM from Scratch: Predicting Baku Property Prices from bina.az Listings

Alimammadova, Nigar Dilavarli, Farid Gojayev, Nihad Karimova, Ulkar

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict property prices in Baku from 64,248 cleaned sale listings scraped from bina.az, as a regression task (price) and as a derived classification task (premium versus standard, split at the median training price). The two core models, a decision tree and a linear soft-margin support-vector machine trained with the Pegasos sub-gradient method, are implemented from scratch in NumPy and compared with scikit-learn under matched settings, the same splits and the same seeds. The cleaning removes two target-leakage columns and a less obvious leak: 36% of the raw rows are repeat scrapes of a listing that is already in the file. Our tree reproduces scikit-learn's to the third decimal (test R^2 0.781 on the log price, F1 0.912 for the tier) and gives the same tier call on 99.9% of held-out listings; our SVM reaches F1 0.849 against 0.852 for `SGDClassifier`, with 95.7% identical calls. The tree beats the linear SVM by six F1 points, because price depends on area and location jointly. An error analysis shows where the models fail: the most expensive 1% of listings, land plots, commercial property and districts with few training listings. One command reproduces every number, table and figure: `github.com/nigar001/ML_project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

I. Introduction

Asking prices on bina.az, the largest property platform in Azerbaijan, depend on size, location, building type and condition in ways that are neither linear nor uniform across the market. We use a public scrape of its sale listings [6] to study two questions with models we wrote ourselves: how well can the price of a listing be predicted (Task A, regression), and can a listing be placed in the upper or lower half of the market (Task B, classification)?

The aim is to understand two standard algorithms from the inside and to compare them honestly with a mature library, not to maximise one score. Our contributions are:

- a documented cleaning pipeline for a raw scraped file, including the removal of target leakage and of repeat scrapes;
- a decision tree (Gini, entropy and variance reduction) and a Pegasos SVM with a random-Fourier-feature extension, both in NumPy only;
- metrics, cross-validation and a benchmark harness written from scratch, with scikit-learn [5] used only as the baseline and to check our numbers;
- an error analysis by price band, property category and district coverage;
- a seeded pipeline in which one command regenerates every table and figure of this report.

II. Background

A decision tree [2] partitions the feature space with axis-aligned threshold splits chosen greedily to reduce an impurity measure, and predicts the majority class or the mean target of the training points in each leaf. It needs no feature scaling and captures interactions, but an unconstrained tree memorises its training data, so depth and leaf size must be limited.

A soft-margin support-vector machine [3] finds a separating hyperplane with a wide margin while tolerating violations. Its primal form is an unconstrained minimisation of a regularised hinge loss, which Pegasos [1] solves by stochastic sub-gradient descent with a step size of $1/(\lambda t)$, with no quadratic-programming solver. A linear SVM can be made non-linear by mapping the inputs to random Fourier features [4], whose inner products approximate a Gaussian kernel.

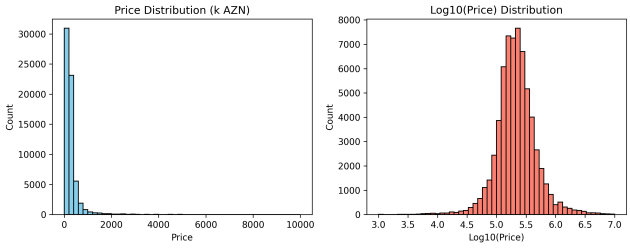


Figure 1. Distribution of the cleaned price and of its base-10 logarithm.

III. Data and Preprocessing

A. The raw file

The dataset [6] (file dated 3 February 2025, downloaded 12 October 2026) has 100,775 rows and 51 columns. All prices are in AZN; 99.7% of the rows are in Baku. It mixes seven property categories: new-build and old-build flats, houses, land, commercial property, offices and garages. Raw prices run from 11 AZN to 600 million AZN and are strongly right-skewed (skewness 252), while their logarithm is close to symmetric (skewness 0.16), see Fig. 1.

B. Leakage

Two columns are arithmetic functions of the target and are dropped: `total_price` equals `price` in 99.99% of the rows, and `unit_price` multiplied by the area reproduces the price to within 0.3%. Features are taken from an explicit list, so identifiers, owner and agency names, the address text and the columns recorded after publication (`views`, `vip`, `featured`) never reach a model.

Table 1. Dataset after cleaning.

Property	Value
Listings after cleaning	64,248
Features	165
Train / validation / test	44,976 / 9,636 / 9,636
Tier threshold (AZN)	210,000
Premium share, train / val / test (%)	49.3 / 49.3 / 49.3

A second leak is not a column. The site was scraped repeatedly, and the file holds only 64,454 distinct listings: 36% of the rows are another scrape of a listing that is already present, some listings up to 17 times. No two rows are fully identical, because each scrape has its own id, so dropping exact duplicates removes nothing. A random split of rows would place copies of one flat in both the training and the test data (48% of the test rows, in our check), and the test score would partly measure memory. We keep one row per listing URL, the most recent scrape.

C. Cleaning decisions

- *Floor.* The text "7 / 9" is split into the floor and the number of floors in the building.
- *Area.* Land is listed in sot and buildings in m^2; sot is converted (1 sot = 100 m^2).
- *Outliers.* We keep prices from 1,000 to 10 million AZN and areas from 15 to 3,000 m^2 (land up to 50,000 m^2). These limits remove 206 of the 64,454 distinct listings: 8 priced below 1,000 AZN (values such as 11 or 250 AZN cannot be the sale price of a property), 63 above 10 million AZN, 36 under 15 m^2, too small for a dwelling, and 99 above the area cap. Land gets a higher cap because plots of several hectares are genuine listings.
- *Missing values.* Rows without a price or an area are dropped. Other missing numbers are set to 0 and missing categories to "Unknown". These are constants, so no statistic of the data, and in particular none of the test split, enters the features.
- *Encoding.* Seven numeric features (area, rooms, floor, floors in the building, land area, latitude, longitude) and six categorical ones (category, repair, bill of sale, mortgage, district, city) are used; the categorical ones are one-hot encoded.

D. Tasks and split

Task A predicts the logarithm of the price. Task B predicts a binary tier: premium if the price is above the median of the *training* prices (210,000 AZN), standard otherwise. The listings are split once, with a fixed seed, into training, validation and test parts of 70/15/15%, stratified so that each part has the same share of premium listings. Table 1 gives the sizes.

IV. Method

A. Decision tree (from scratch)

For a node with class proportions p_k the tree uses the Gini impurity or the entropy, and for regression the variance of the target:

$$G = 1 - \sum_k p_k^2, \quad H = -\sum_k p_k \log_2 p_k, \quad I_{\text{reg}} = \tfrac{1}{N}\sum_i (y_i - \bar{y})^2. \tag{1}$$

A split of a parent node into children c with N_c points is scored by its impurity decrease,

$$\Delta I = I(\text{parent}) - \sum_c \tfrac{N_c}{N} I(c), \tag{2}$$

and the tree takes the feature and threshold with the largest decrease. Candidate thresholds are the midpoints between consecutive distinct sorted values of a feature. Growth is recursive and stops at `max_depth`, below `min_samples_split` points, when a child would have fewer than `min_samples_leaf` points, or when the best decrease is below `min_impurity_decrease`. A leaf predicts the majority class (with class frequencies as probabilities) or the mean target. The importance of a feature is the total impurity decrease of the splits that use it, weighted by the share of training points reaching them and normalised to sum to one.

The split search sorts each feature once per node and obtains the impurity of every candidate threshold from cumulative class counts (classification) or cumulative sums of y and y^2 (regression), so a node costs $O(n \log n)$ per feature instead of $O(n^2)$; a test checks this fast search against a plain reference that scans every threshold. Features are visited in a seeded random order and the first best threshold is kept, so ties between equally good splits are broken reproducibly, though not in scikit-learn's order. A midpoint that rounds onto the upper of two neighbouring values falls back to the lower one, so no child is ever empty, and, as in scikit-learn, `min_impurity_decrease` is compared with the decrease weighted by the node's share of all training rows. Unit tests check the impurities against hand calculations, that the tree finds the same splits as scikit-learn on a small fully-determined example, that each of the four stopping settings changes the tree, and that `predict_proba` returns leaf frequencies.

B. Support-vector machine (from scratch)

With labels $y_i \in \{-1, +1\}$ the linear soft-margin SVM minimises

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big). \tag{3}$$

Pegasos [1] draws a random example i at step t, sets $\eta_t = 1/(\lambda t)$ and updates

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\,\mathbf{w} + \eta_t\, y_i \mathbf{x}_i\, \mathbf{1}\big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1\big]. \tag{4}$$

The margin width is $2/\|\mathbf{w}\|$, and the support vectors are the training points with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1$. The hinge loss is sensitive to scale, so features are standardised with the mean and standard deviation of the training rows only. Our implementation folds the bias into $\mathbf{w}$ by appending a constant feature (so b is regularised too), averages the sub-gradient over mini-batches of 32 examples drawn with replacement, projects $\mathbf{w}$ onto the ball $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$ after each step as in [1], and trains for 20 passes over the training rows. The objective

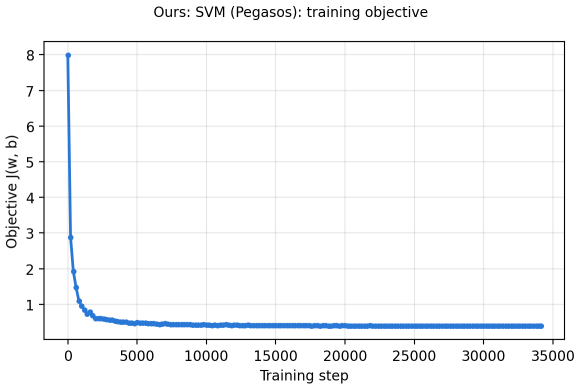


Figure 2. Training objective J of our SVM (Task B, final fit).

is recorded every 200 steps; Fig. 2 shows it falling and flattening.

For the non-linear extension we use random Fourier features [4], $z(\mathbf{x}) = \sqrt{2/D}\cos(\mathbf{W}\mathbf{x}+\mathbf{u})$ with Gaussian $\mathbf{W}$ and uniform $\mathbf{u}$, and run the same linear solver on $z(\mathbf{x})$. Unlike kernelised Pegasos this keeps one weight vector instead of one coefficient per training point, which matters with tens of thousands of listings. The wider SVM study (`svm_experiments.py`, run by the same command) uses $D = 500$ and sweeps the bandwidth γ together with λ on the validation split. The best random-feature model reached a validation F1 of 0.809, against 0.825 for the linear SVM in the same study, so the extension did not pay off on these features and the linear SVM remains our main model. That study works on a reduced feature set of 54 columns and its own split, so its numbers are not comparable digit by digit with the tables below.

V. Experimental Setup

Metrics. Task A is scored by RMSE, MAE and R^2 on the log price, and by MAE in AZN after transforming predictions back. Task B is scored by precision, recall and F1 for the premium class, by accuracy, and by ROC-AUC computed from ranks with tied scores sharing their average rank. The metrics are our own NumPy code; tests check them against hand-worked examples and against scikit-learn to twelve significant digits.

Baselines and matched settings. Our tree is compared with `DecisionTreeRegressor` and `DecisionTreeClassifier` given the same criterion and the same four stopping settings. Our SVM is compared with `SGDClassifier` with hinge loss and $\alpha = \lambda$, which minimises the same objective (3) with the same kind of solver; its features are standardised inside a scikit-learn `Pipeline`. Table 2 lists the settings, which come from the validation study. *Tuning* (`src/tuning.py`) uses the training and validation parts only. For the SVM we sweep $\lambda \in \{10^{-5}, \ldots, 10^{-1}\}$ with 20 passes each; validation F1 peaks at $\lambda = 10^{-3}$ (0.840). For the tree, on a seeded sample of 20,000 training rows, we sweep `max_depth` from 2 to 15 for Gini and entropy (Task B) and for variance (Task A), then `min_samples_leaf` $\in \{1, 5, 20, 50\}$ at the chosen depth. Gini and entropy stay within 0.01 F1 of each other at every depth; Gini peaks slightly higher and is kept. One set of stopping settings serves both tasks, so the depth is chosen on Task A, the tree's main task. The resulting curves are in Fig. 3.

Table 2. Settings used for every reported number.

Setting	Value
Tree: `max_depth`	15
Tree: `min_samples_split`	2
Tree: `min_samples_leaf`	20
Tree: `min_impurity_decrease`	0.0
Tree: criterion (Task B)	gini
SVM: λ	0.001
Task A target	log(price)

Protocol. Nothing from held-out rows is used for a decision. (i) The tier threshold is the median of the training prices and is fixed before any model is fitted. (ii) Settings are chosen on the validation part. (iii) Uncertainty comes from 5-fold stratified cross-validation on the training and validation rows; inside each fold the threshold and the scaling statistics are recomputed from that fold's training part. (iv) Each model is then refitted on the training and validation rows and scored once on the test part. All randomness is seeded with 42, and two runs of the pipeline give identical numbers. Times are wall-clock seconds on one laptop (Intel Core i7-13650HX).

VI. Results

A. Headline results

Tables 3 and 4 give the test-split metrics of every model, with the cross-validated mean and standard deviation of the main metric. Our tree reproduces scikit-learn's to the third decimal on both tasks: test R^2 0.781 for both on the log price, and F1 0.912 with AUC 0.969 for both on the tier. Our SVM is within 0.003 F1 of `SGDClassifier`. The tree is the stronger model for the tier (F1 0.912 against 0.849), and its cross-validated scores vary by less than 0.01 across folds.

Fig. 4 shows the ROC curves on the test split. The tree is ahead of the linear SVM over almost the whole range: the tier depends on area and location jointly, which a tree expresses with a few splits and a single hyperplane cannot.

B. Agreement with scikit-learn

Table 5 compares each of our models with its library counterpart on one held-out fold. Exact agreement is not expected. Two trees can differ when two splits have the same impurity decrease, because each implementation breaks the tie in its own order (scikit-learn visits features in a seeded random order), and because scikit-learn stores features as 32-bit floats, which moves thresholds in the last digits. Two SVMs trained by stochastic steps stop near the optimum of (3), not at it, so we compare the objective value they reach. The trees grow to the same depth with almost the same number of leaves and share the root split: area ≤ 100.25 m^2 for the price and ≤ 92.15 m^2 for the tier, which scikit-learn stores as 92.1500015 because of its 32-bit floats. They give the same tier on 99.87% of held-out listings, and their price predictions differ by 0.005 on the log scale on average; the few larger gaps come from the handful of nodes where a tie or

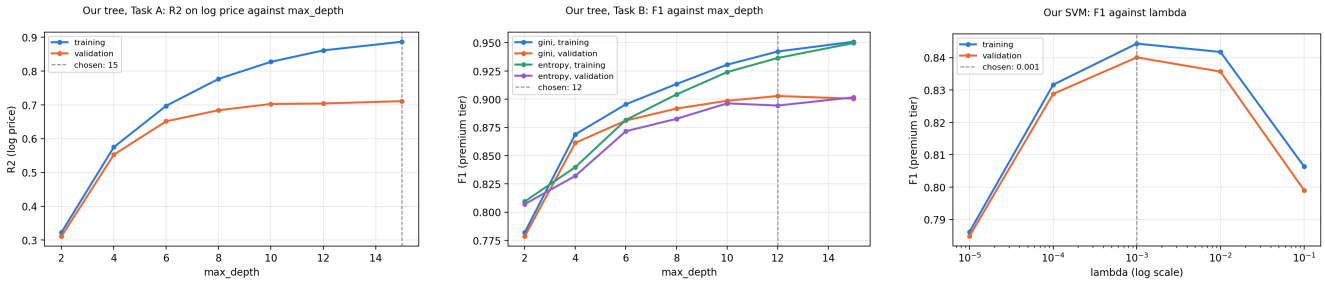


Figure 3. Validation curves on the validation split. Left: our tree, Task A, R^2 against depth. Middle: our tree, Task B, F1 against depth for Gini and entropy (the dashed line marks Task B's own best depth). Right: our SVM, F1 against λ.

Table 3. Task A, price regression on the log price: test split, and cross-validated R^2 (mean $\pm$ std over 5 folds).

Model	RMSE	MAE	R^2	MAE (AZN)	CV R^2
Ours: tree	0.365	0.203	0.781	79,181	0.769 ± 0.007
sklearn: DecisionTreeRegressor	0.365	0.203	0.781	79,219	0.769 ± 0.007

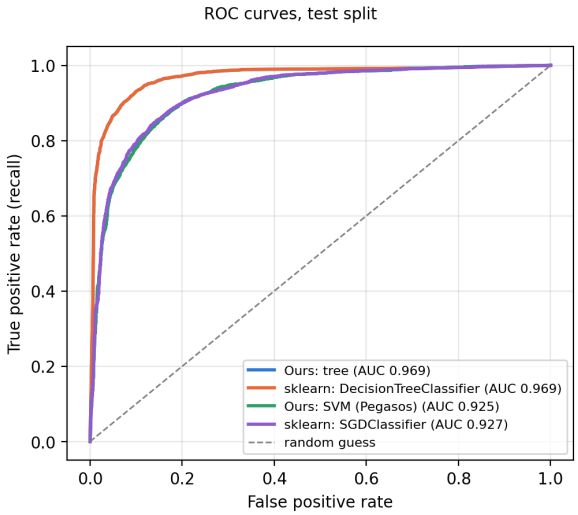


Figure 4. ROC curves on the test split, Task B.

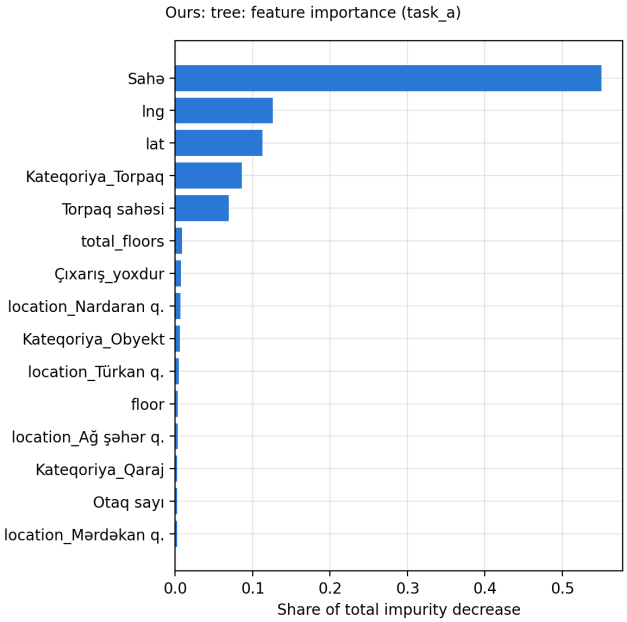


Figure 5. Feature importances of our tree, Task A.

a rounded threshold sends a listing to a different leaf. The SVMs agree on 95.7% of listings. Ours reaches $J = 0.392$ against 0.387: `SGDClassifier` runs until its own convergence test, while ours stops after a fixed 20 passes, slightly short of the optimum.

C. Margin and support vectors

Table 6 reports the learned $\|\mathbf{w}\|$, the margin width and the number of support vectors. The classes overlap heavily near the median price, so a large share of the training points lies on or inside the margin.

D. What the models use

Fig. 5 shows the impurity-decrease importances of the tree for Task A, and Fig. 6 the largest weights of the linear SVM on standardised features. The tree relies mostly on the area and then on the coordinates; the one-hot district columns add little once latitude and longitude are available. The SVM cannot split on coordinates, so besides rooms, area and category it puts weight on individual districts instead.

E. Speed

Table 7 gives fit and predict times at three training sizes. scikit-learn's tree is compiled code, while ours is NumPy driven from Python, so a large constant-factor gap is expected. Our tree fits about ten times more slowly than scikit-learn's (2.3 s against 0.21 s on 20,000 rows for Task B), but its time grows almost linearly with the number of rows (0.11, 0.51 and 2.3 s for 1,000, 5,000 and 20,000 rows), as expected from sorting each feature once per node. Our SVM fits 20,000 rows in 0.9 s.

VII. Discussion and Limitations

A. Where the models fail

Tables 8 and 9 split the test error by price band, property category and the number of training listings in the listing's district; the band edges and the district counts come from the

Table 4. Task B, price tier: test split, and cross-validated F1 (mean ± std over 5 folds).

Model	Accuracy	Precision	Recall	F1	AUC	CV F1
Ours: tree	0.914	0.919	0.905	0.912	0.969	0.909 ± 0.002
sklearn: DecisionTreeClassifier	0.914	0.918	0.905	0.912	0.969	0.909 ± 0.002
Ours: SVM (Pegasos)	0.851	0.848	0.851	0.849	0.925	0.841 ± 0.007
sklearn: SGDClassifier	0.853	0.847	0.857	0.852	0.927	0.846 ± 0.005

Table 5. Our models against scikit-learn on one held-out fold.

Pair	Predictions	Ours	scikit-learn
Tree, Task A	mean gap 0.0053, same root split	depth 15, 1,175 leaves	depth 15, 1,178 leaves
Tree, Task B	99.9% same calls, same root split	depth 15, 664 leaves	depth 15, 665 leaves
SVM, Task B	95.7% same calls	$J = 0.3923$, $\|\mathbf{w}\| = 3.428$	$J = 0.3875$, $\|\mathbf{w}\| = 3.249$

Table 6. Linear SVMs on the training and validation rows.

Model	$\|\mathbf{w}\|$	$2/\|\mathbf{w}\|$	Support vectors	% of points
Ours: SVM (Pegasos)	3.357	0.596	20,957	38.4
sklearn: SGDClassifier	3.255	0.614	20,563	37.7

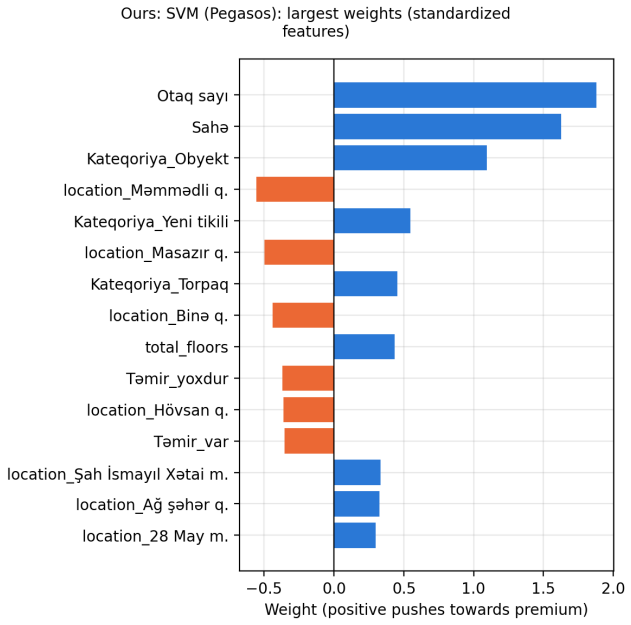


Figure 6. Largest weights of our linear SVM (standardised features). Positive weights push towards the premium tier.

training rows only.

The luxury tail. The error on the most expensive 1% of listings is several times the error in the middle of the market. A tree of limited depth has a limited number of leaves, so rare expensive listings share a leaf with cheaper ones and are pulled towards their mean; the largest misses in Table 10 are dominated by under-predictions of very expensive land and commercial property.

Property type. Flats are numerous and alike, and are predicted well. Land and commercial property are few, are priced by things the table does not hold (zoning, frontage, business use), and their area is on a different scale from that of flats.

Sparse districts. Error falls steadily as the number of training listings in a district grows, in both tasks.

The tier boundary. In Task B the mistakes concentrate in the two price bands next to the median: a listing a few thousand AZN from the threshold is close to a coin flip for any model, while the cheapest and the most expensive listings are classified far more reliably.

B. Bias and variance

Fig. 3 shows the usual pattern. Shallow trees (depth 2–4) score low on both training and validation data: high bias. As depth grows the training score keeps rising while the validation score flattens after depth 10, and the widening gap is the tree starting to memorise its training rows: variance. Raising `min_samples_leaf` from 1 to 20 lowers the training R^2 from 0.95 to 0.80 but keeps the validation R^2 at its best, so larger leaves remove variance at no cost; at 50 both fall. For the SVM a very small λ gives a narrow margin and a lower validation F1 (0.785 at 10^{-5}), and a very large one a wide margin that underfits (0.799 at 10^{-1}). Note that these sweeps use a 20,000-row sample, so their absolute scores sit below the final ones.

C. Limitations

- Prices are asking prices from listings, not sale prices.
- One model serves seven property types. A model per type, or dropping land and commercial property, would probably help both tasks.
- Missing numbers are set to 0. A tree can isolate that value, but a linear model reads it as a real floor or land area.
- Land plots of up to 50,000 m^2 give the area a very long tail, which hurts the linear SVM after standardising; a logarithm of the area would be the first thing to try.
- The one-hot columns are created from all listings before the split. This uses no target information, but a district seen only in the test part still gets a column.
- Repeat scrapes are matched by listing URL; a property re-listed under a new URL counts as a new listing.
- Cross-validation folds share most of their training data, so the reported ± describes stability across folds, not a confidence interval.

Table 7. Fit and predict time in seconds by number of training rows.

Model	Fit @ 1,000	Predict @ 1,000	Fit @ 5,000	Predict @ 5,000	Fit @ 20,000	Predict @ 20,000
Ours: tree (task_a)	0.0921	0.0013	0.4887	0.0077	1.7923	0.0433
sklearn: DecisionTreeRegressor (task_a)	0.0065	0.0005	0.0386	0.0018	0.1926	0.0061
Ours: tree (task_b)	0.1128	0.0020	0.5141	0.0126	2.3204	0.0578
sklearn: DecisionTreeClassifier (task_b)	0.0055	0.0005	0.0417	0.0017	0.2100	0.0060
Ours: SVM (Pegasos) (task_b)	0.0416	0.0001	0.2064	0.0003	0.8961	0.0007
sklearn: SGDClassifier (task_b)	0.0179	0.0011	0.0981	0.0039	0.3845	0.0146

Table 8. Task A: mean absolute error on the log price, test split.

Group	*n*	Our tree	sk tree
By price band (training quantiles)			
bottom 10%	1,023	0.373	0.373
10-50%	3,862	0.144	0.143
50-90%	3,798	0.166	0.166
90-99%	863	0.366	0.367
top 1%	90	0.849	0.849
By property category			
New-build flat	5,548	0.132	0.132
Old-build flat	1,778	0.123	0.123
House	1,439	0.294	0.294
Land	495	0.755	0.755
Commercial	354	0.564	0.566
Office	13	0.438	0.438
Garage	9	0.451	0.451
By training listings in the district			
under 50	173	0.437	0.436
50-499	2,217	0.307	0.307
500 or more	7,246	0.166	0.166

Table 9. Task B: share of wrong tier calls, test split.

Group	*n*	Our tree	sk tree	Our SVM	sk SGD
By price band (training quantiles)					
bottom 10%	1,023	0.033	0.033	0.052	0.050
10-50%	3,862	0.090	0.090	0.174	0.178
50-90%	3,798	0.113	0.112	0.171	0.166
90-99%	863	0.028	0.028	0.064	0.054
top 1%	90	0.000	0.000	0.011	0.022
By property category					
New-build flat	5,548	0.074	0.074	0.155	0.155
Old-build flat	1,778	0.082	0.082	0.090	0.088
House	1,439	0.107	0.106	0.149	0.143
Land	495	0.125	0.125	0.212	0.204
Commercial	354	0.175	0.175	0.266	0.251
Office	13	0.000	0.000	0.000	0.000
Garage	9	0.000	0.000	0.000	0.000
By training listings in the district					
under 50	173	0.116	0.116	0.173	0.133
50-499	2,217	0.097	0.097	0.158	0.148
500 or more	7,246	0.083	0.082	0.145	0.147

- The data are from early 2025 and almost entirely from Baku.

VIII. Conclusion

Both models, written from scratch in NumPy, match scikit-learn on a real, messy dataset: our tree to the third decimal, our Pegasos SVM within 0.003 F1, with the gaps explained by tie-breaking, float precision and the optimiser's stopping rule. A depth-15 tree predicts Baku asking prices to within about 14% for ordinary flats and classifies the price tier with F1 0.91, while a linear SVM is limited by the joint effect of area and location. Both fail on what is rare: the luxury tail, land, commercial property and thinly covered districts. Next we would train a model per property type, use the logarithm of the area with proper missing-value indicators, and build a random forest from our own trees.

Tools and Acknowledgements

AI assistance. F. Dilavarli designed and led the evaluation side of the project and coordinated the team's hand-overs. He used Claude Code (Anthropic) as a coding assistant for the implementation and for drafting the report and slides, and reviewed its output; the commits concerned carry a co-author line. N. Alimammadova used Claude as an AI assistant to draft the SVM code, experiments and tests and to explain the method. N. Gojayev used Claude (Anthropic) to review the tree, to write the faster split search, the tests and the comparison script, and to draft report text; he ran the code and checked the outputs. U. Karimova used Claude (Anthropic) as an AI assistant to build the demo user interface (`ui/app.py`); she ran it and checked its output on the real data.

Data and software. The dataset is the Bina.az Sale Project scrape published on Kaggle [6]. We used NumPy, pandas and matplotlib, and scikit-learn [5] for the baselines and for checking our metrics only.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[4] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, 2007, pp. 1177–1184.

Table 10. The ten largest price errors on the test split, Task A.

Category	Area (m^2)	Actual (AZN)	Predicted (AZN)
Land	50,000	7,000,000	138,790
Land	50,000	8,500,000	1,767,076
Commercial	330	7,200,000	1,289,074
Commercial	800	8,000,000	2,441,780
Commercial	550	6,400,000	854,860
Land	28,000	5,600,000	477,660
Commercial	570	5,700,000	1,043,973
Commercial	100	5,100,000	821,294
Commercial	1,330	7,000,000	2,936,983
Commercial	170	4,500,000	457,233

[5] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] "Bina.az Sale Project," Kaggle dataset published by user `sehriyarmemmedli`, file dated Feb. 2025. [Online]. Available: `https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project`